

PA1 — HTTP Web Server

CS 158A Computer Networks | Fall 2026 | SJSU

Overview

In PA1 you build a working HTTP web server in Python — one that a real browser can connect to and retrieve files from. Part A has you implement core HTTP parsing and response-building functions, and debug two functions that are provided but deliberately broken. Part B has you implement the two connection-handling strategies HTTP actually uses: non-persistent (HTTP/1.0 style) and persistent (HTTP/1.1 keep-alive). Part C reuses Part B code to run a live experiment showing why concurrent servers exist at all — you'll observe, measure and report the differences between a single-threaded vs. multi-threaded server.

What you need from PA0

Your virtual environment from PA0 should still be active. No additional packages are required. You will also need Wireshark installed and working.

Files for this assignment

- server.py
- client.py
- www/index.html
- www/about.html

Place all files in a single folder. The www/ folder must be in the same directory as server.py. Part C requires one additional file, www/large.bin, which you will generate yourself — see Part C below. Do not download or request this file from anywhere; you create it locally with a single command.

File Structure

After downloading and unzipping the PA1 files, your folder should look like this before you generate large.bin:

```
pa1/
  server.py      <- 2 TODOs (A.1) + 2 seeded bugs to fix (A.2) + 2 TODOs (Part B)
  client.py      <- boilerplate + slow_client() and fast_client() (Part C)
  www/
    index.html   <- sample file your server will serve
    about.html   <- second sample file
```

Activate your virtual environment before running anything:

```
# Windows
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate
```

Part A.1 — Core Parsing and Status Functions

Implement these two functions from scratch. They have no dependencies on anything else in the file.

`get_status_line(code)`

Return the full HTTP/1.1 status line for a given integer status code. This is the first line of every HTTP response.

Parameters

- code — int — the HTTP status code (e.g. 200, 404, 405)

Returns

- str — e.g. "HTTP/1.1 200 OK" or "HTTP/1.1 404 Not Found"

Examples

- `get_status_line(200)` -> "HTTP/1.1 200 OK"
- `get_status_line(404)` -> "HTTP/1.1 404 Not Found"
- `get_status_line(405)` -> "HTTP/1.1 405 Method Not Allowed"

Required codes

- 200 OK
- 400 Bad Request
- 404 Not Found
- 405 Method Not Allowed
- 500 Internal Server Error

parse_http_request(raw_data)

Everything sent/received to/from the network is raw bytes. Decode an incoming raw byte string received from the socket and extract the HTTP method, path, version, and headers. This is the entry point for every request your server receives.

Parameters

- raw_data — bytes — raw bytes from `socket.recv()`

Returns

- dict with keys 'method', 'path', 'version', 'headers' — or None if parsing fails

A raw request looks like this over the wire:

```
GET /index.html HTTP/1.1\r\n
Host: 127.0.0.1:8080\r\n
Connection: keep-alive\r\n
\r\n
```

Hint

- Decode raw_data to a string.
- Split on the appropriate HTTP delimiters and/or spaces to parse the information from HTTP header
- Generate and return dict with method, path, version and headers
- Return None if raw_data cannot be parsed

Part A.2 — Debug Two Seeded Bugs

`serve_file()` and `build_http_response()` are provided fully implemented in `server.py`. Each one contains exactly one bug that was put there deliberately. Your job is not to rewrite these functions — it is to run the server, observe what breaks, trace it to the specific line responsible, and fix only that line.

This is graded in two parts: the fix itself (Part A.2 in the grading table), and a short written account of the debugging process, which goes in your executive summary and is also worth points on its own (see "Debug narrative" in the grading table).

serve_file(path) — has one bug

Look up a file in the `www/` directory and return its contents. Read through the provided implementation before running anything. Try to get `index.html` via `curl` or web browser. Was it successful? What went wrong? When you hit it, the symptom will show up in your browser or in `serve_file()`'s return value, not as a crash. Wireshark is your friend here to know what the server actually sent to the browser.

Signature

- tuple: (status_code, content_bytes, mime_type) — status 200 if found, 404 if not

build_http_response(status_code, body_bytes, extra_headers=None) — has one bug

Assembles a complete HTTP response as bytes. Follow the response structure described below. The bug here is quieter than the one in `serve_file()` — again, try to get `index.html` via `curl -v` (and inspect via Wireshark). Do you see the full contents on the terminal? What went wrong?

Response structure

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 412\r\n
Date: Sat, 28 Mar 2026 18:00:00 GMT\r\n
Server: PA1Server/1.0\r\n
Connection: keep-alive\r\n
\r\n
<body bytes>
```

Part B — Connection Modes

Implement the two connection handlers from scratch. Use your Part A work to build out the logic for these functions. The difference between them is the lifetime of the TCP connection — and this same code is what Part C runs under a third server mode, so a bug you leave here will show up again in Part C.

handle_connection_non_persistent(conn)

Every request from the client requires a new connection. This is HTTP/1.0 behavior.

Flow

```
recv data -> parse -> serve_file -> build_response -> send -> close
```

handle_connection_persistent(conn)

A TCP connection can serve multiple requests in sequence. The connection closes only when the client sends `Connection: close`, disconnects, or the `TIMEOUT` is reached. This is HTTP/1.1 keep-alive behavior.

Loop

```
set timeout -> recv -> parse -> serve_file -> check Connection header
-> if 'close': send with Connection: close, break
-> otherwise : send with Connection: keep-alive, loop
```

Exit conditions

- `socket.timeout` raised | `recv` returns `b''` | client sends `Connection: close`

Note on the timeout

- Call `conn.settimeout(TIMEOUT)` at the top of the loop, before each `recv()`.
- Wrap `recv()` in a `try/except socket.timeout` block — catch it and break. This prevents the server from hanging forever on a silent client.
- Parse the request — send 400 and break if parsing fails and only accept GET (send 405 and break for other methods)
- Handle ‘connection’ header value ‘close’ appropriately (sending ‘keep-alive’ and loop again as needed)
- Make sure to close connection after loop exit

Part C — Concurrency Experiment

`server.py` includes a third mode, single-threaded, that is already fully implemented for you — there is nothing to write in `server.py` for Part C. In this mode, the server calls your Part B connection handler directly in its main accept loop, with no

threading.Thread involved at all. That means the server cannot accept a second connection until the first one's handler completely finishes.

You will use two clients — one deliberately slow, one fast — to make this concrete. Against a single-threaded server, the fast client should have to wait behind the slow one. Against your own multi-threaded persistent handler from Part B, it should not.

Step 0 — Generate the large test file

The slow client needs a large file to request — large enough that the server's attempt to send it can genuinely block, rather than completing instantly. This file is NOT included in your PA1 download; generate it yourself, once, before running the experiment:

```
# macOS / Linux
dd if=/dev/urandom of=www/large.bin bs=1M count=5
```

```
# Windows (PowerShell)
fsutil file createnew www\large.bin 5242880
```

This creates a 5 MB file at www/large.bin. Do not include large.bin in what you submit — see Submission below.

slow_client(host, port, path) — your work

Connect to the server, send a GET request for a large object, and read the response back **DELIBERATELY SLOWLY** — simulating a client that is still receiving data (a slow network, a struggling device, a user who paused the download) rather than one that has finished (hint: use `SLOW_READ_CHUNK_SIZE` and sleep between reads for `SLOW_READ_DELAY` seconds). Also, print a progress line **INCLUDING** `_ts()`, but only about once per `PRINT_INTERVAL` seconds of elapsed time — not once per chunk to avoid too much scrolling. After final read, print a final summary line **INCLUDING** `_ts()`: total bytes received and total elapsed time. Make sure to close the socket.

fast_client(host, port, path) — your work

This client requests a small file and times how long the full response takes to arrive. The logic here is much shorter than `slow_client()`, since it just times a normal request using the provided `read_http_response()` helper. Hint: use `time.time()` to timestamp before sending a request and after a full response is received and print the elapsed time in seconds.

Designing and Running the experiment

First, think about the experiment objective (showing benefits of concurrent multi-threaded web server vs. non-current single-threaded web server; which client will likely impact response time for other clients in a single threaded server situation?) and how you would design the experiment to showcase this objective. Hint: You'd need to run the server, slow client and fast client in 3 different terminals. Which client would you start first, and why? Note that you'll have to run the clients within a gap of a few seconds (much less than whatever is the `TIMEOUT` value at server; why is that?)

You need two terminal windows for the clients, plus a third for the server — three terminals total, all open at the same time.

- Run the server(s)/client(s) a few times to confirm your code works and hypothesis checks out before starting the final run for evidence capture.
- Run the single threaded experiment a few times to validate your hypothesis. Record your evidence (see below) once you are satisfied.
- Repeat the same experiment with a persistent multi-threaded server. Record your evidence once you are satisfied.
- Take your required screenshots (see Part C Evidence below) during both runs — you cannot go back and re-capture a run after it's finished, so have Wireshark or your screenshot tool ready before you start each run.

If your terminal scrolls past what one screenshot can capture

With the required once-per-second progress printing, this should not normally happen within the few seconds this experiment takes. If it does anyway (e.g. you changed the timing constants while experimenting):

- Widen or lengthen your terminal window before starting the run, so more lines fit on screen.
- If that didn't help, take 2 screenshots (instead of one) showing the begin and end of the terminal output.

Evidence Requirements — Applies to All Screenshots in This Assignment

The following four rules apply to every screenshot you submit in this assignment — Wireshark captures and Part C terminal screenshots alike. They are stated once here rather than repeated under every individual instruction.

- Full application window. Screenshot the entire Wireshark window or terminal window, not a cropped excerpt of just the relevant line or column. A crop showing only the piece being asked about is not sufficient — the surrounding context is part of the evidence.
- Highlighting is required, not optional. The specific value, line, or field being referenced must be visibly marked (circled, boxed, or highlighted) directly in the image itself.
- Write your explanation after capturing the evidence, describing what that specific screenshot actually shows — not a prediction of what should happen. If your actual result differs from what you expected, describe what you actually observed and take your best attempt at explaining why.
- All evidence — every screenshot, every caption, the debug narrative, all of it — goes into the single summary.pdf. Do not submit separate image files.

One more general note: your machine is not a standalone lab system. Background processes generate their own network traffic. If a Wireshark filter returns more than you expect, narrow it (e.g. `tcp.port == 8080`, since that's the port your own server listens on) so you're only capturing this assignment's own traffic.

Wireshark Evidence — Required

Capture traffic while your fixed, working server is running and include three screenshots in your executive summary, each with a caption. Caption requirement: each caption must reference a specific value visible in that screenshot — a byte count, a sequence number, a status code — and connect it to your code. A caption that could apply to any screenshot (e.g. "here are the headers") will not receive credit.

Capture 1 — Raw HTTP headers

A GET request/response pair with both sets of headers expanded. Show at minimum the status line, Content-Type, and Content-Length on the response side.

Capture 2 — The 404 response

Request `missing.html` and capture the response. Note in your caption whether this is the same capture you used while debugging `serve_file()`, or a fresh one taken after the fix.

Capture 3 — Connection reuse, persistent vs non-persistent

This capture evidence of your Part B work specifically. Start a Wireshark capture, then with your server running in persistent mode, send two requests over one connection using `curl`:

```
curl -v --http1.1 http://127.0.0.1:8080/index.html http://127.0.0.1:8080/about.html
```

Stop the capture and apply a filter scoped to your server's port (see the note above). How many TCP handshakes do you see (SYN, SYN-ACK, ACK)? Screenshot this, along with the two HTTP exchanges visible and highlighted.

Then restart your server in non-persistent mode. Start a NEW, separate Wireshark capture (stop the old one first) before repeating the same `curl` command — don't keep the first capture running across both server restarts, or you'll have to dig through one long capture to find the right packets instead of having two clean, separate screenshots. This time how many TCP handshakes do you see? Screenshot this too, along with the two HTTP exchanges visible and highlighted

Caption must state the number of handshakes you observed in each capture and connect that directly to what `handle_connection_persistent()` and `handle_connection_non_persistent()` each do differently.

Part C Evidence — Required

These are terminal screenshots — the effect Part C demonstrates along with time elapsed for the HTTP request/response for the slow and fast clients in each situation. Include screenshots of all terminals in both server modes with appropriate captions.

All four Evidence Requirements rules above still apply: full window, highlighted, captioned after the fact, all embedded in summary.pdf.

Caption requirement for all four

Each caption must quote the actual timestamp(s) or wait-time value visible in that specific screenshot — not a value from a different run, and not a rounded or approximate restatement. Since server.py and client.py both timestamp their output with the same wall-clock format, your caption for Screenshots 1 and 2 (and separately for 3 and 4) should show that you can trace one specific real-time gap across both terminals, not just report two unrelated numbers.

Testing Your Server

Work through these checks in order. Each one tests a specific function.

Browser test

- Start the server: `python server.py`
- Open a browser and navigate to: `http://127.0.0.1:8080/index.html`
- You should see the PA1 index page load.
- Navigate to `http://127.0.0.1:8080/missing.html` — you should get a 404 response.
- Navigate to `http://127.0.0.1:8080/about.html` — the about page should load.

Terminal test with curl

curl shows you the raw HTTP headers, which is how you will catch the `build_http_response()` bug:

```
curl -v http://127.0.0.1:8080/index.html
```

Look for the status line, Content-Type, Content-Length, and Connection headers in the output. Compare the reported Content-Length against the actual file size on disk.

Executive Summary

The executive summary is a required part of your submission. Apart from evidence for the 3 parts, it should also include answering the questions below directly. Keep answers grounded in your specific implementation — reference your code, your variable names, and your actual observed output.

Length and format

- Embed all screenshots, each with its caption.
- Submit as a PDF alongside your .py files.

Debug Narrative — Required, one entry per Part A.2 bug

Q1 and Q2 – for each of the two seeded bugs (`serve_file`, `build_http_response`), include:

- The exact error text, browser behavior, or curl -v output you saw before fixing it.
- One or two sentences on the root cause — the specific line and why it was wrong.
- Confirmation output after the fix (working page load, or curl -v showing correct Content-Length).

Q3 — Part A recap

Q3. Walk through what your server returns when a client requests `index.html` versus `missing.html`. Name the specific functions in your code that produce each outcome and what they return.

Q4 + Q5 — Part B

Q4. In your `handle_connection_persistent()` function, what specific condition causes the loop to exit and the connection to close? Refer to the exact variable or exception in your code.

Q5. What would happen to your persistent handler if a client connected but never sent any data and never closed the connection? How does your current code handle this, and what is the `TIMEOUT` value you used?

Submission

Submit the following files

- `server.py` — your completed implementation (2 written functions + 2 debugged functions + 2 written handlers)
- `client.py` — your completed implementation (`slow_client` and `fast_client`)
- `summary.pdf` — your executive summary (2–3 pages of writing) including the debug narrative, all screenshots with captions, and answers to questions above (Q1–Q5).

Do not submit the `www/` folder, and do not submit `www/large.bin` specifically — it's a 5 MB generated file with no content of its own, and it's trivially regenerated with the one command in Part C. Submit only the two Python files and the PDF.

Grading

Component	Points
Part A.1 — <code>get_status_line()</code> and <code>parse_http_request()</code>	15
Part A.2 — Debug <code>serve_file()</code> and <code>build_http_response()</code>	10
Part B — Non-persistent and persistent connection handlers	15
Part C — <code>slow_client()</code> , <code>fast_client()</code> and experiment design	13
Debug narrative — error evidence + root cause for both Part A.2 bugs	15
Wireshark evidence — 3 captures + captions	15
Part C evidence — timestamped terminal screenshots + captions	10
Q3 — Part A recap	3
Q4 + Q5 — Part B (loop exit condition, silent client)	4
Total	100